Lesson 8 Teacher's Key: Under Pressure

Unreal Engine (optional advanced track) - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

A second Event Dispatcher (OnEnemyEscaped) added to BP_Enemy alongside the existing OnEnemyDestroyed, with its handler living on BP_GameManager - the Blueprint-native version of the same two-signal / two-dispatcher pattern seen in Godot and Unity this lesson, reinforcing that both the pattern AND the dedicated-manager structure are cross-engine principles, not engine-specific tricks.

Level 1 - Full Worked Solution

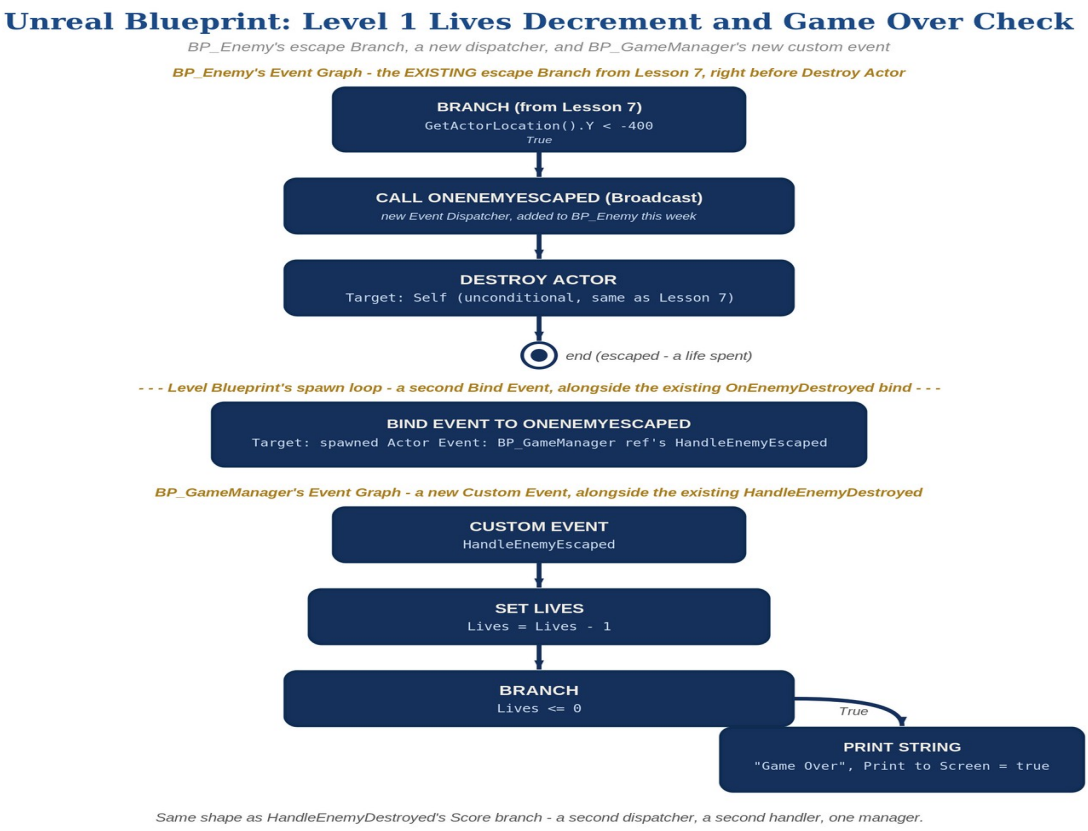
- Lives is a new variable on BP_GameManager, parallel to EnemiesRemaining - same placed Actor, same pattern, different variable, different dispatcher responsible for changing it.
- Call OnEnemyEscaped is added to the EXISTING escape Branch in BP_Enemy's Event Graph, right before the pre-existing Destroy Actor node - one new node in an already-correct location.
- The Level Blueprint's spawn loop binds BOTH dispatchers per spawned enemy to the SAME BP_GameManager reference (OnEnemyDestroyed to HandleEnemyDestroyed from Lesson 6, OnEnemyEscaped to HandleEnemyEscaped new this week) - two independent bindings, one dedicated manager Actor holding both handlers.

Highlighted lines are new this week; everything else is code you already have.

```
BP_Enemy's Event Graph (escape Branch, revised):
Event Tick ->
  Branch(GetActorLocation().Y < -400) -> True:
    Call OnEnemyEscaped (Broadcast)
    Destroy Actor(Self)

Level Blueprint spawn loop (alongside binding OnEnemyDestroyed to the same
BP_GameManager reference):
  Bind Event to OnEnemyEscaped(Target = spawned Actor, Event =
BP_GameManager ref's HandleEnemyEscaped)

BP_GameManager:
Custom Event HandleEnemyEscaped ->
  Lives = Lives - 1
  Branch(Lives <= 0) -> True: Print String("Game Over", Print to Screen =
true)
```



BP_Enemy's escape Branch, a new dispatcher, and BP_GameManager's new custom event, in Unreal Blueprint - the node-for-node equivalent of the Level 1 code.

Why This Structure

- This is the same principle as Godot's two-signal design and Unity's two-static-method design, expressed in Unreal's dispatcher/delegate idiom: two separate broadcasts, two separate handlers, two separate pieces of state, all owned by BP_GameManager, with no shared hook to misuse.
- This is the direct payoff of Lesson 7's structure AND this week's Rule 13 retrofit: because the escape Branch was already the single, unambiguous home for 'this enemy left the play area unshot,' and BP_GameManager was already the single home for shared game state, adding a new dispatcher call and a new handler required zero changes to the overlap event's hit-detection logic or to EnemiesRemaining.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (Unreal): import both files as Sound Wave assets. In BP_GameManager's HandleEnemyEscaped: Lives = Lives - 1 -> Play Sound 2D(sfx_lifelost.wav) -> Branch(Lives <= 0) -> True: Play Sound 2D(sfx_gameover.wav) -> Print String("Game Over"). C++ path: UGameplayStatics::PlaySound2D with the same two assets.
- For students who did both Godot and Unreal (or who are comparing engines): ask them to name the equivalent piece in each engine (signal/emit vs. dispatcher/broadcast, GameManager node vs. BP_GameManager Actor) - a good vocabulary-building exercise that reinforces the underlying concepts are engine-independent.

- Level 3 structure: point out that BP_Enemy now has exactly two dispatchers, cleanly named for what they mean (Destroyed, Escaped), and BP_GameManager now holds exactly two pieces of shared state (EnemiesRemaining, Lives) - contrast this with a hypothetical single generic 'OnEnemyGone' dispatcher or a Lives variable left on the Level Blueprint, and ask why either would be worse.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: shooting an enemy never changes lives (only the kill path
changes enemies_remaining);
// an escape never changes enemies_remaining (only the escape path changes
lives);
// "Game Over" appears once, when lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
BP_GameManager - Level 3:
Custom Event HandleEnemyEscaped ->
  Lives = Lives - 1
  Play Sound 2D(sfx_lifelost.wav)
  Print String("Lives: " + Lives, Print to Screen = true)
  Branch(Lives <= 0) -> True:
    Play Sound 2D(sfx_gameover.wav)
    Print String("Game Over", Print to Screen = true)
```

Common Student Mistakes

- Calling OnEnemyEscaped from the successful-Cast (hit) branch, or OnEnemyDestroyed from the escape Branch - either one causes cross-contamination between Lives and EnemiesRemaining.
- Adding Lives and HandleEnemyEscaped to the Level Blueprint instead of BP_GameManager, out of habit from before the Lesson 7 GameManager retrofit - watch for this specifically, since it's the most likely regression this week.
- Forgetting to bind OnEnemyEscaped in the Level Blueprint's spawn loop (only OnEnemyDestroyed bound, copied from Lesson 6 without adding the new binding node) - Lives never changes even though the dispatcher fires correctly.
- Both Print String messages appearing at the same screen position - adjust one's position/duration so "Game Over" and "You Win!" don't overlap.

Alternate Path Notes: C++ (code, optional advanced track)

ACppEnemy.h declares FOnEnemyEscaped as a second multicast delegate, alongside the existing FOnEnemyDestroyed - same mechanism, different name, different meaning.

ACppEnemy::Tick()'s escape branch calls OnEnemyEscaped.Broadcast() right before Destroy() - one new line in an already-correct location, mirroring how OnEnemyDestroyed.Broadcast() is called from OnOverlap().

The GameManager actor class's BeginPlay() binds a second UFUNCTION to each spawned enemy's FOnEnemyEscaped delegate using AddDynamic(), tracking a new Lives member variable alongside the existing EnemiesRemaining - the same class, the same binding pattern, a second piece of state.

```
// ACppEnemy.h - add alongside the existing delegate:
DECLARE_MULTICAST_DELEGATE(FOnEnemyEscaped);
FOnEnemyEscaped OnEnemyEscaped;

// ACppEnemy.cpp - Tick()'s escape branch (revised):
if (GetActorLocation().Y < -400.f) {
    OnEnemyEscaped.Broadcast();
    Destroy();
}

// GameManager's handler:
int32 Lives = 3;

void AGameManager::HandleEnemyEscaped() {
    Lives--;
    if (Lives <= 0) {
        GEngine->AddOnScreenDebugMessage(-1, 5.f, FColor::Red, TEXT("Game
Over"));
    }
}
```

- This mirrors the Blueprint path's structure exactly: a second delegate, broadcast from the same escape branch, handled by a second method on the same GameManager class that already owns EnemiesRemaining - same principle as BP_GameManager, expressed in C++.